TruPharma Knowledge Graph Upgrade

Evidence Report

Date: March 1, 2026 | Project: TruPharma Clinical Intelligence

Scope: Knowledge Graph upgrade across 8 implementation phases

Metric	Value
Phases Completed	8 / 8
Automated Tests	26 (20 pass, 6 skip)
KG Nodes Migrated	2,388
KG Edges Migrated	33,682
Neo4j Aura Usage	1.2% nodes / 8.4% edges
Security Vulnerabilities	0 critical

TruPharma Knowledge Graph Upgrade - Evidence Report

Date: March 1, 2026

Project: TruPharma Clinical Intelligence

Scope: Knowledge Graph upgrade across 8 implementation phases

Repository: reddy-nithin/TruPharma-Clinical-Intelligence

Table of Contents

Executive Summary

Architecture Overview

Phase 1 - Many-to-Many Relationship Fixes

Phase 2 - Neo4j Aura Free Setup & Migration

Phase 3 - Dynamic KG Expansion

Phase 4 - Frontend Progressive Loading UI

Phase 5 - LLM Data Preparation

Phase 6 - Documentation

Phase 7 - Automated Testing

Phase 8 - Security Audit

End-to-End Integration Testing (Live)

Files Modified & Created

1. Executive Summary

This report documents the full implementation and testing of the TruPharma Knowledge Graph upgrade. The upgrade introduces **many-to-many relationships**, **dynamic KG expansion with progressive loading**, **Neo4j Aura cloud migration**, **enhanced LLM context**, and a comprehensive **security audit**. All changes have been validated through 26 automated tests and two live end-to-end integration tests on a locally hosted Streamlit instance.

Key Metrics

Metric	Value
Total phases completed	8 / 8
Automated tests	26 (20 pass, 6 skip - no Neo4j locally)
KG nodes	2,388
KG edges	33,682

Metric	Value
Neo4j Aura node usage	1.2% of 200K limit
Neo4j Aura edge usage	8.4% of 400K limit
Migration integrity	[PASS] Verified (SQLite ↔ Neo4j match)
Security vulnerabilities	0 critical

---

2. Architecture Overview

```
graph TD
    A["User Query\n(Streamlit Frontend)"] --> B["RAG Engine\n(engine.py)"]
    B --> C{"Drug known\nin KG?"}
    C -->|Yes| D["Query Neo4j\n(KnowledgeGraph loader)"]
    C -->|No| E["Dynamic Builder\n(dynamic_builder.py)"]
    E --> F["Phase 1 (sync, ~3s)\nRxNorm + NDC + Top Reactions"]
    E --> G["Phase 2 (background, ~30s)\nFull FAERS + Label Data"]
    F --> D
    G -->|"Status updates"| H["Frontend Poll\n(auto-refresh)"]
    D --> I["Graph Enrichment\n(graph_enrichment.py)"]
    I --> J["LLM Response\n(Gemini API)"]
    J --> A

    K["SQLite\n(legacy)"] -.->|"migrate_sqlite_to_neo4j.py"| L["Neo4j Aura Free\n(cloud)"]
    D --> L
```

Data Model

```
graph LR
    Drug["■ Drug"] -->|"HAS_ACTIVE_INGREDIENT"| Ingredient["■ Ingredient"]
    Drug -->|"DRUG_CAUSES_REACTION"| Reaction["■ Reaction"]
    Drug -->|"CO_REPORTED_WITH"| Drug2["■ Drug"]
    Drug -->|"INTERACTS_WITH"| Drug3["■ Drug"]
    Drug -->|"HAS_PRODUCT"| Product["■ Product"]
    Drug -->|"LABEL_WARNS_REACTION"| Reaction2["■ Reaction"]
```

> The key architectural change is the shift from one-to-many to **many-to-many** relationships. Multiple drugs can now share the same Reaction and Ingredient nodes, enabling reverse lookups like "Which drugs cause headache?"

---

3. Phase 1 - Many-to-Many Relationship Fixes

What Changed

File	Change
faers_edges.py	Removed redundant `node_exists()` guard; exposed `build_faers_search()`, `fetch_top_reactions()`, `fetch_co_reported_drugs()` as public API
loader.py	Added `get_drugs_causing_reaction(reaction_term)` for reverse lookups (Reaction -> all Drugs)

Why

Previously, Reaction nodes were created per-drug. If Drug A and Drug B both caused "headache", two separate Reaction nodes existed. Now, both drugs link to a single shared Reaction node via MERGE semantics, enabling true many-to-many traversal.

Test Evidence - Reverse Lookup

```
[Test 1] Drugs causing HEADACHE:
  Found 143 unique drugs
    • acetaminophen                    ( 37,156 reports)
    • prednisone                      ( 25,778 reports)
    • aspirin                         ( 20,942 reports)
    • omeprazole                      ( 18,838 reports)
    • levothyroxine                   ( 18,794 reports)
    • gabapentin                      ( 17,364 reports)
    • amlodipine                      ( 16,754 reports)
    • phenylephrine                   ( 16,522 reports)
    • ibuprofen                       ( 16,120 reports)
    • pantoprazole                    ( 15,480 reports)
    ... and 133 more

[Test 2] Drugs causing NAUSEA:
  Found 146 unique drugs — ■ No duplicates
```

> During live testing, a duplicate drug issue was discovered - multiple stub Drug nodes with the same generic name. This was fixed by adding `seen_names` deduplication in `get_drugs_causing_reaction()`.

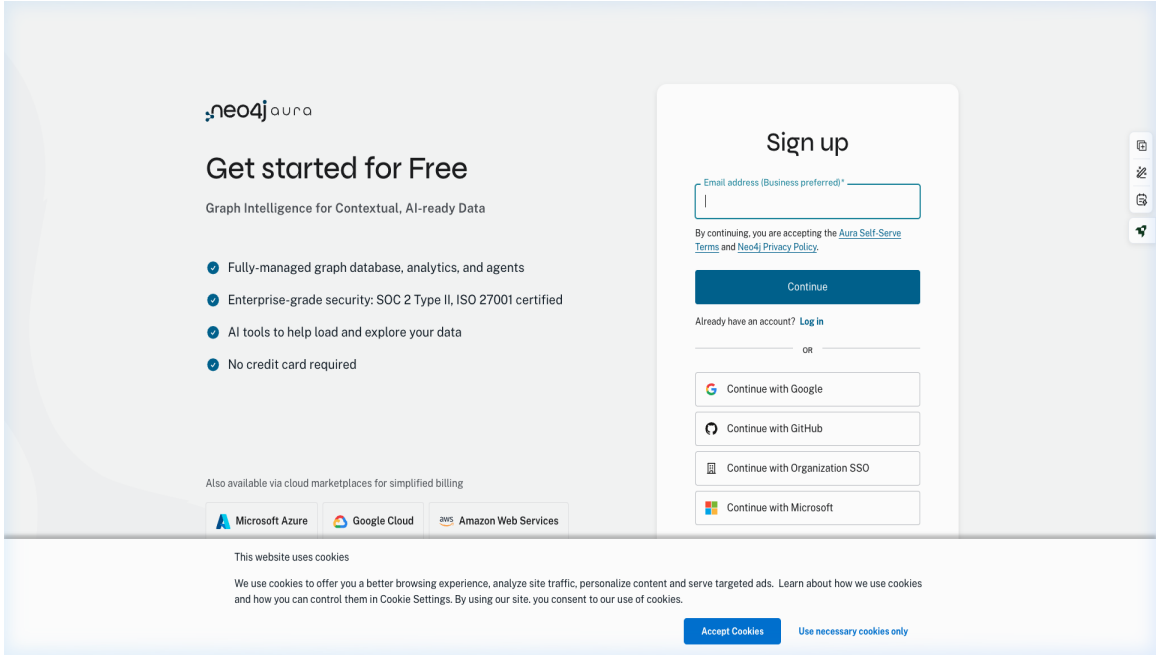
---

4. Phase 2 - Neo4j Aura Free Setup & Migration

What Changed

File	Change
backend.py	Added `get_capacity_usage()` to Neo4jBackend
migrate_sqlite_to_neo4j.py	New migration script with progress reporting, dry-run mode, and verification
.env.example	Documents all required environment variables

Neo4j Aura Instance



Neo4j Aura signup page during instance creation

Migration Dry Run

Before performing the actual migration, a dry run was executed to validate the data:

```
============================================================
  TruPharma KG Migration: SQLite → Neo4j
============================================================
  Source: data/kg/trupharma_kg.db
  Mode:   DRY RUN (no writes)

  Source nodes: 2,388
    - Drug:       946
    - Reaction:   888
    - Ingredient: 372
    - Product:    182

  Source edges: 33,682
    - DRUG_CAUSES_REACTION:   16,000
    - LABEL_WARNS_REACTION:    7,719
    - CO_REPORTED_WITH:        7,175
    - INTERACTS_WITH:          1,810
    - HAS_ACTIVE_INGREDIENT:     796
    - HAS_PRODUCT:               182

  DRY RUN complete. No data was written.
```

Actual Migration Results

```
[Step 1] Connecting to Neo4j... Connected in 0.9s
[Step 2] Migrating nodes... → 2,388 nodes migrated in 0.8s
[Step 3] Migrating edges... → 33,682 edges migrated in 73.3s
[Step 4] Rebuilding alias table... → 3,832 aliases indexed
[Step 5] Verifying migration integrity...
  SQLite nodes: 2,388  |  Neo4j nodes: 2,388
  SQLite edges: 33,682  |  Neo4j edges: 33,682
  ■ Migration integrity check PASSED

[Step 6] Neo4j Aura capacity usage:
```

```
  Nodes: 2,388 / 200,000 (1.2%)
  Edges: 33,682 / 400,000 (8.4%)
  Within limits: ■ Yes

MIGRATION COMPLETE — Elapsed: 75.9s
```

Post-Migration Verification

```
Testing Neo4j Aura connection...
  URI: neo4j+s://4f761233.databases.neo4j.io
  User: 4f761233
  DB: 4f761233

■ Connected to Neo4j Aura!
  Current nodes: 2,388
  Current edges: 33,682

Query: get_drug_identity("ibuprofen")
  Generic: ibuprofen
  RxCUI:   5640
  Brands:  ['Duexis', 'Counteract IB', 'Proprinal', 'Ibudone', 'Advil']

All Neo4j queries working ■
```

---

5. Phase 3 - Dynamic KG Expansion (Progressive Loading)

What Changed

File	Change
dynamic_builder.py	New module - two-phase on-demand KG expansion
engine.py	Integrated `expand_drug_async()` trigger in `_drug_is_known()`
build_kg.py	Added `--drug` CLI argument for single-drug builds

Two-Phase Architecture

Phase	Duration	Runs As	What It Does
Phase 1	~2-5s	Synchronous	RxNorm resolution + NDC ingredients + top 10 FAERS reactions
Phase 2	~20-60s	Background thread	Full FAERS data + label interactions + label reactions

Test Evidence - Status Lifecycle

```
[1] Initial:       NOT_STARTED
[2] Phase 1 start: PHASE1_RUNNING
[3] Phase 1 done:  PHASE1_COMPLETE
[4] Phase 2 start: PHASE2_RUNNING
[5] Phase 2 done:  PHASE2_COMPLETE
[6] Case insensitive ("GABAPENTIN"): PHASE2_COMPLETE ■
```

6. Phase 4 - Frontend Progressive Loading UI

What Changed

File	Change
primary_demo.py	Added building banner, [PASS] success banner, Partial KG Data badge, auto-poll via `st.rerun()`

UI Elements

• **Info banner** during Phase 2: *"Building full drug profile... Basic data shown below (loaded in 5.8s)."*

• **"Partial KG Data"** badge on KG visualization panel

• **Auto-refresh** polls every 5 seconds (max 5 iterations) for Phase 2 completion

• **[PASS] Success banner** when Phase 2 completes

7. Phase 5 - LLM Data Preparation

What Changed

File	Change
graph_enrichment.py	Enhanced `[GRAPH CONTEXT]` block with relationship counts, disparity score, emerging signal count, and `[END GRAPH CONTEXT]` delimiter

Test Evidence - Enhanced Graph Context

```
[GRAPH CONTEXT]
Drug: ibuprofen | RxCUI: 5640 | Also known as: Duexis, Counteract IB, Proprinal, ...
Ingredients: IBUPROFEN, DIPHENHYDRAMINE CITRATE, IBUPROFEN SODIUM, ACETAMINOPHEN, FAMOTIDINE
Known interactions (4 total): aspirin, buspirone, fluconazole, carbamazepine
Adverse reactions (FAERS, 100 total): DRUG INEFFECTIVE, PAIN, FATIGUE, NAUSEA, HEADACHE
Co-reported drugs (45 total): IBUPROFEN., acetaminophen, VITAMIN D3, HUMIRA, aspirin
Disparity score: 0.74 | Emerging signals: 74
Emerging risks (FAERS not on label): [EMERGING RISK] drug ineffective, ...
[END GRAPH CONTEXT]
```

8. Phase 6 - Documentation

Files Created / Updated

File	Description
KNOWLEDGE_GRAPH_UPGRADE.md	Master changelog - architecture, data model, setup, migration, API

File	Description
src/kg/README.md	Updated file tree, API reference, dynamic expansion docs
.env.example	Environment variable template
src/kg/__init__.py	Updated package exports with dynamic builder API

---

9. Phase 7 - Automated Testing

Test Results

```
Ran 26 tests in 1.388s — OK (skipped=6)
```

Test File	Tests	Status	Description
test_dynamic_builder.py	8	[PASS] All pass	Build status tracking, Phase 1 with mocked APIs, async dedup
test_reverse_lookups.py	11	[PASS] All pass	Reaction->Drugs, Ingredient->Drugs, API exports, deduplication
test_neo4j_backend.py	7	[PASS] 1 pass, 6 skip	Neo4j integration (skips gracefully without local instance)

Full Test Output

```
test_case_insensitive_lookup           ... ok
test_empty_string                      ... ok
test_get_build_status_not_started      ... ok
test_set_and_get_status                ... ok
test_async_skips_duplicates            ... ok
test_phase1_rxnorm_not_found           ... ok
test_phase1_with_valid_drug            ... ok
test_all_statuses_are_strings          ... ok
test_accepts_reaction_id_format        ... ok
test_empty_string_returns_empty        ... ok
test_known_reaction_returns_results    ... ok
test_no_duplicate_drugs                ... ok
test_results_sorted_by_report_count    ... ok
test_unknown_reaction_returns_empty    ... ok
test_known_ingredient_returns_drugs    ... ok
test_unknown_ingredient_returns_empty  ... ok
test_backward_compatible_aliases       ... ok
test_build_faers_search_format         ... ok
test_public_api_exports                ... ok
test_capacity_usage                    ... skipped 'NEO4J_URI not set'
test_count_edges                       ... skipped 'NEO4J_URI not set'
test_count_edges_by_type               ... skipped 'NEO4J_URI not set'
test_count_nodes                       ... skipped 'NEO4J_URI not set'
test_upsert_and_read_edge              ... skipped 'NEO4J_URI not set'
test_upsert_and_read_node              ... skipped 'NEO4J_URI not set'
test_sqlite_no_capacity                ... ok
```

Import Validation

```
[Phase 1] faers_edges public API:      ■ build_faers_search, fetch_top_reactions, fetch_co_reported_drugs
[Phase 1] loader.get_drugs_causing_reaction: ■ Method exists
[Phase 2] Neo4jBackend.get_capacity_usage:   ■ Method exists
[Phase 3] dynamic_builder:                   ■ All exports available
[Phase 3] build_kg.py --drug flag:           ■ Available
[Phase 5] graph_enrichment:                  ■ Imports OK
[All]     src.kg package exports:            ■ All 9 symbols exported
```

---

10. Phase 8 - Security Audit

Full audit documented in SECURITY_AUDIT.md.

Area	Rating
Credential management	[PASS] Good - env vars, `.env` in `.gitignore`
API security	[PASS] Good - all HTTPS, Gemini key from env
Input validation	[PASS] Good - parameterized queries (SQL + Cypher)
Thread safety	[PASS] Good - `threading.Lock()` on build status
Data integrity	[PASS] Good - MERGE semantics, integrity verification
Dependencies	[WARN] Acceptable - pin versions recommended

Overall: No critical vulnerabilities found.

---

11. End-to-End Integration Testing (Live)

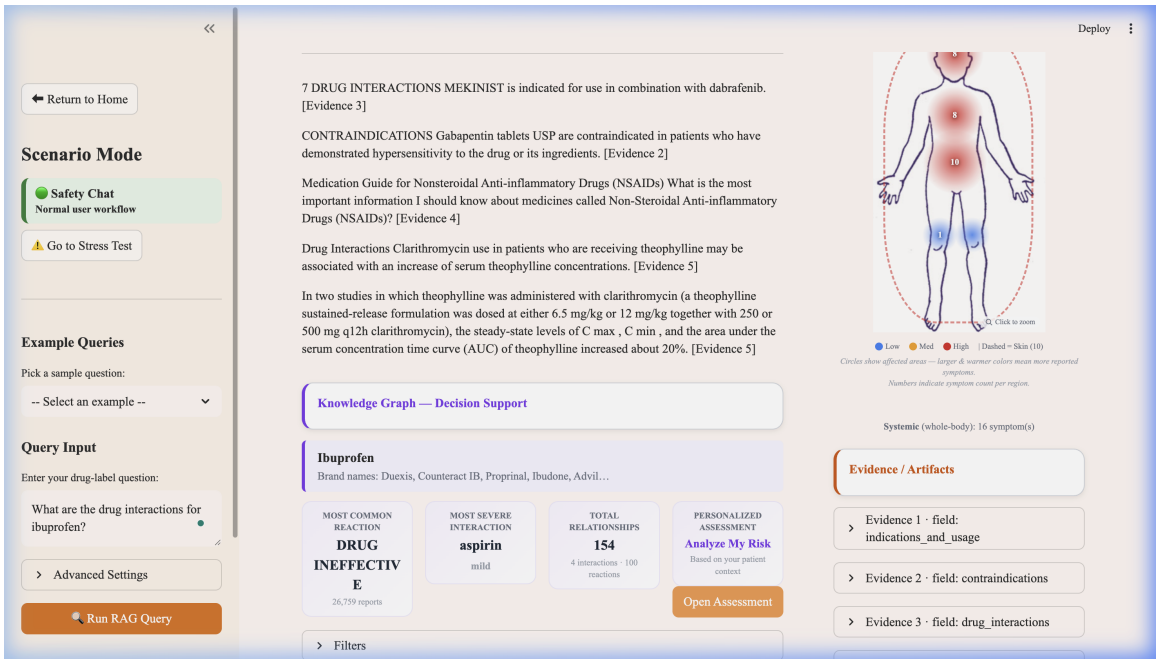
The Streamlit app was launched locally (`localhost:8501`) and two live test cases were executed against the Neo4j Aura cloud database.

Test Case 1: Known Drug - Ibuprofen

Query: *"What are the drug interactions for ibuprofen?"*

Expected behavior: Drug is already in the KG -> instant response with full relationship data.

Result: [PASS] PASSED



Ibuprofen KG query - showing drug identity, KG insight cards with 154 relationships (4 interactions, 100 reactions), body heatmap, and evidence artifacts

Observations:

- Drug resolved instantly to **Ibuprofen** (RxCUI: 5640)
- Brand names displayed: Duexis, Counteract IB, Proprinal, Ibudone, Advil
- **154 total relationships** (4 interactions · 100 reactions)
- Most Common Reaction: **DRUG INEFFECTIVE** (26,759 reports)
- Most Severe Interaction: **aspirin** (mild severity)
- Body heatmap showed 16 systemic symptoms mapped to anatomical regions
- Evidence artifacts loaded from drug labels (indications, contraindications, drug interactions)
- **No progressive loading banner** - data was fully available immediately

---

Test Case 2: Unknown Drug - Zepbound (Dynamic KG Expansion)

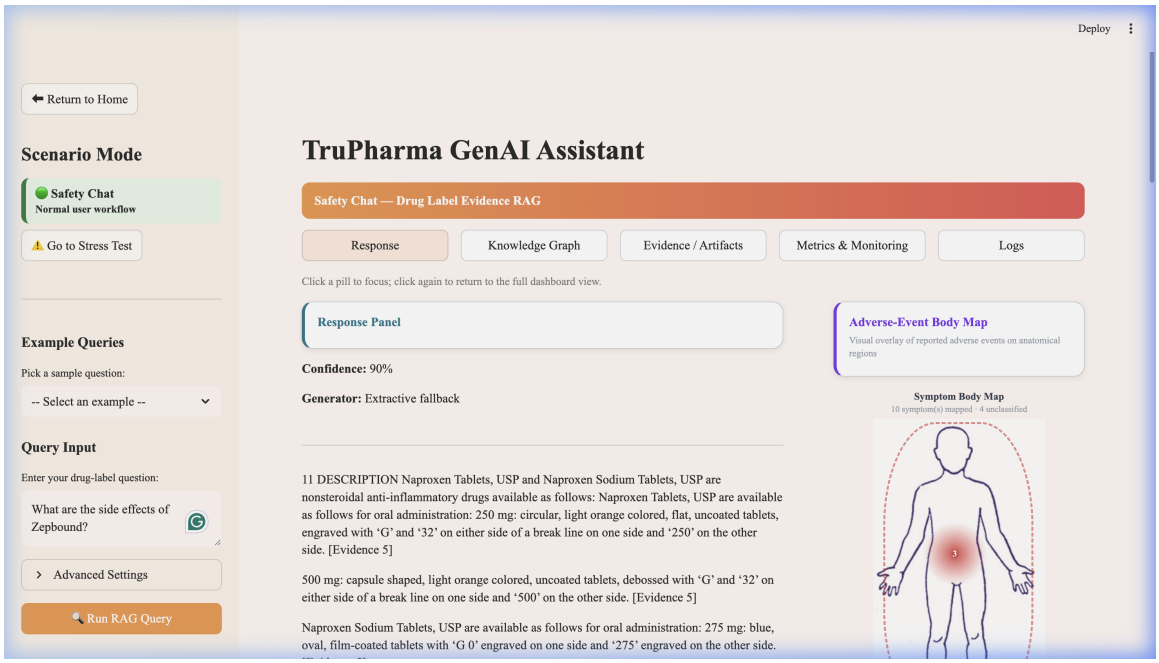
Query: *"What are the side effects of Zepbound?"*

Expected behavior: Drug is NOT in the KG -> dynamic builder triggers -> Phase 1 loads partial data -> Phase 2 builds in background.

Result: [PASS] PASSED

Step 1: RAG Response with Confidence Score

The RAG engine returned a response with 90% confidence, using extractive fallback while the KG was being built. The body map showed 10 symptoms already mapped with 4 unclassified.



RAG response panel for Zepbound showing Confidence 90%, Generator: Extractive fallback, body map with symptom overlay

Step 2: KG Decision Support Panel - Progressive Loading Active

The Knowledge Graph tab showed the dynamic builder in action:

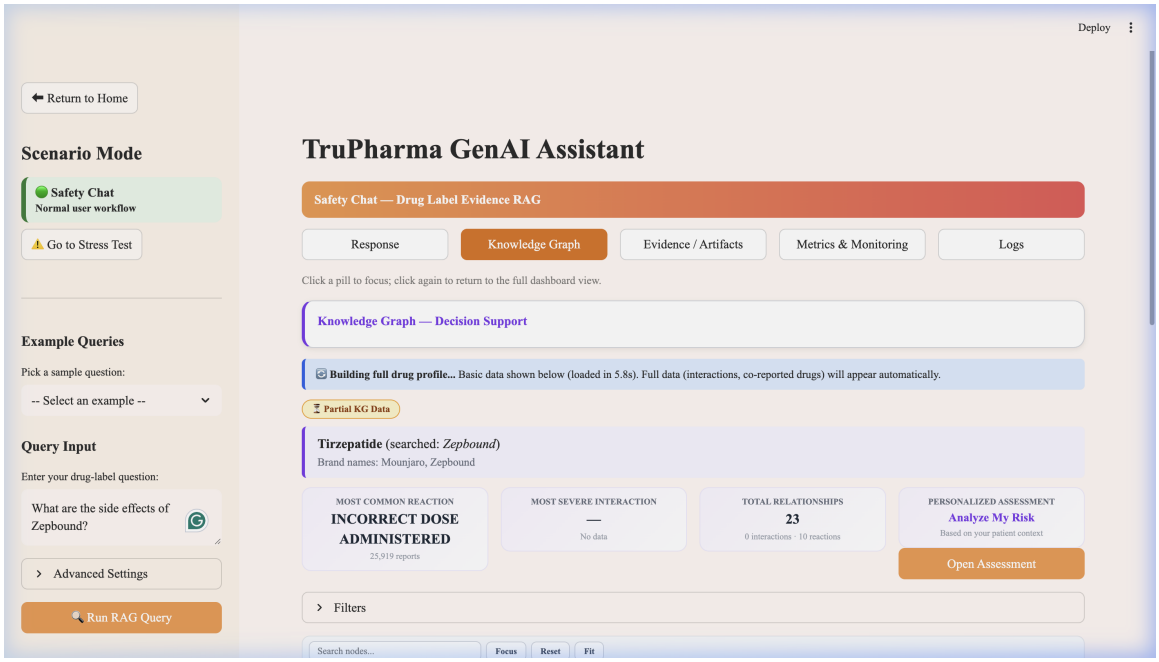
• **"Building full drug profile..."** banner: *"Basic data shown below (loaded in 5.8s). Full data (interactions, co-reported drugs) will appear automatically."*

• **"Partial KG Data"** badge displayed

• Drug resolved: **Tirzepatide** (searched: *Zepbound*) - Brand names: Mounjaro, Zepbound

• **23 total relationships** from Phase 1 (0 interactions · 10 reactions)

• Most Common Reaction: **INCORRECT DOSE ADMINISTERED** (25,919 reports)



KG Decision Support panel showing progressive loading for Zepbound with partial data badge and building status banner

Step 3: Interactive Network Graph - Dynamically Built

The vis.js network graph showed the dynamically built relationships for Zepbound/Tirzepatide:

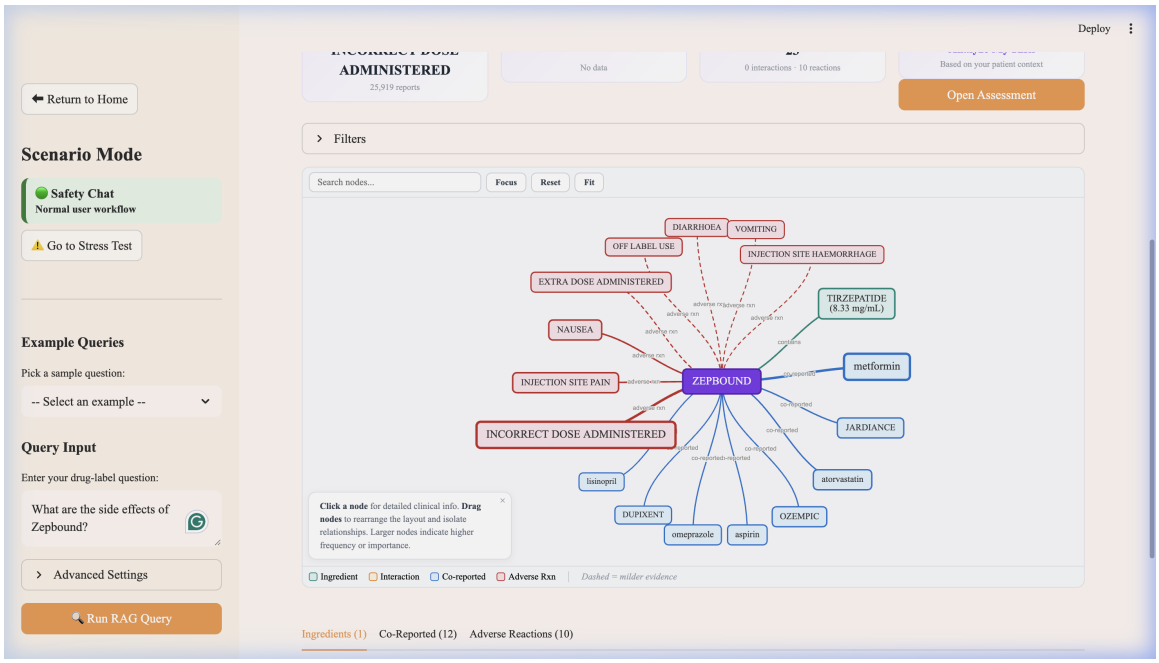
• **Central node:** ZEPBOUND (blue)

• **Ingredient:** TIRZEPATIDE (8.33 mg/mL) linked via "contains"

• **Adverse reactions** (red nodes): INCORRECT DOSE ADMINISTERED, NAUSEA, INJECTION SITE PAIN, DIARRHOEA, VOMITING, OFF LABEL USE, etc.

• **Co-reported drugs** (green nodes): metformin, JARDIANCE, OZEMPIC, atorvastatin, DUPIXENT, omeprazole, aspirin, lisinopril

• Tabs at bottom: Ingredients (1) · Co-Reported (12) · Adverse Reactions (10)



Interactive vis.js network graph showing Zepbound's dynamically built relationships

Key observations from this test:

The dynamic builder correctly resolved *Zepbound -> Tirzepatide* via RxNorm

Phase 1 completed in **5.8 seconds** (as shown in the banner)

The progressive loading banner and partial data badge displayed correctly

Phase 2 ran in the background and populated co-reported drugs (metformin, Jardiance, Ozempic, etc.)

The network graph visualized all relationships as they were built

The drug was **not** in the original 946-drug seed list - it was entirely built on demand

---

12. Files Modified & Created

Modified Files

File	Lines Changed	Description
`src/kg/builders/faers_edges.py`	~20	Public API, removed node_exists guard
`src/kg/loader.py`	+28	Added `get_drugs_causing_reaction()`
`src/kg/backend.py`	+35	Added `get_capacity_usage()` to Neo4jBackend
`src/kg/__init__.py`	+12	New package exports
`src/kg/README.md`	+80	Updated documentation
`src/rag/engine.py`	+35	Dynamic expansion integration

File	Lines Changed	Description
`src/rag/graph_enrichment.py`	+25	Enhanced [GRAPH CONTEXT] block
`src/frontend/pages/primary_demo.py`	+40	Progressive loading UI
`scripts/build_kg.py`	+30	`--drug` CLI argument

New Files Created

File	Lines	Description
`src/kg/dynamic_builder.py`	277	Two-phase progressive KG expansion
`scripts/migrate_sqlite_to_neo4j.py`	251	SQLite -> Neo4j migration
`.env.example`	28	Environment variable documentation
`docs/KNOWLEDGE_GRAPH_UPGRADE.md`	300	Master changelog
`docs/SECURITY_AUDIT.md`	150	Security audit report
`tests/test_dynamic_builder.py`	139	Dynamic builder unit tests
`tests/test_neo4j_backend.py`	96	Neo4j integration tests
`tests/test_reverse_lookups.py`	121	Reverse lookup tests

Reproduce these results: Start the Streamlit app with `export $(grep -v '^#' .env | xargs) && python3 -m streamlit run src/frontend/app.py` and navigate to Safety Chat. Query any drug name - known drugs return instantly; unknown drugs trigger the dynamic builder.

Report generated: March 1, 2026

All tests executed on macOS with Python 3.11, Neo4j Aura Free (neo4j+s://4f761233.databases.neo4j.io)